

MODELO PADRAO DE ENTREGA DE ATIVIDADES

Etapa	Entrega final	Atividade	10
Encontro	11	Curso	ADS / ULBRA
Aluno	Vinicius Casagrande	Professor(a)	Caroline Pacheco da Rosa
Projeto	Recomendacao de treinos personalizados em academia		

Artefato verificavel

Repositorio (GitHub): https://github.com/Vinicasagrande87/fundamentos_ia

Notebook desta etapa: atividade10_academia.ipynb (15 secoes, executado do inicio ao fim no Google Colab, com a figura da matriz de confusao visivel).

Slides finais da AP2: entregas/AP2_Final_ViniciusCasagrande.pptx (11 slides, versao final).

README.md atualizado: secao "10. Atividade 10 - Avaliacao do modelo (entrega final)" e backlog do projeto.

1. Contexto

Na Atividade 9 foi treinado o primeiro modelo (DecisionTreeClassifier, max_depth=5) e comparado com o baseline. Esta atividade aprofunda a avaliacao desse mesmo modelo: metricas por classe, matriz de confusao visual, significado dos erros no dominio do projeto, escolha justificada da metrica principal, teste de overfitting e verificacao adicional de resultado bom demais (duplicatas de perfil e ordem cronologica).

2. Metricas de classificacao (conjunto de teste)

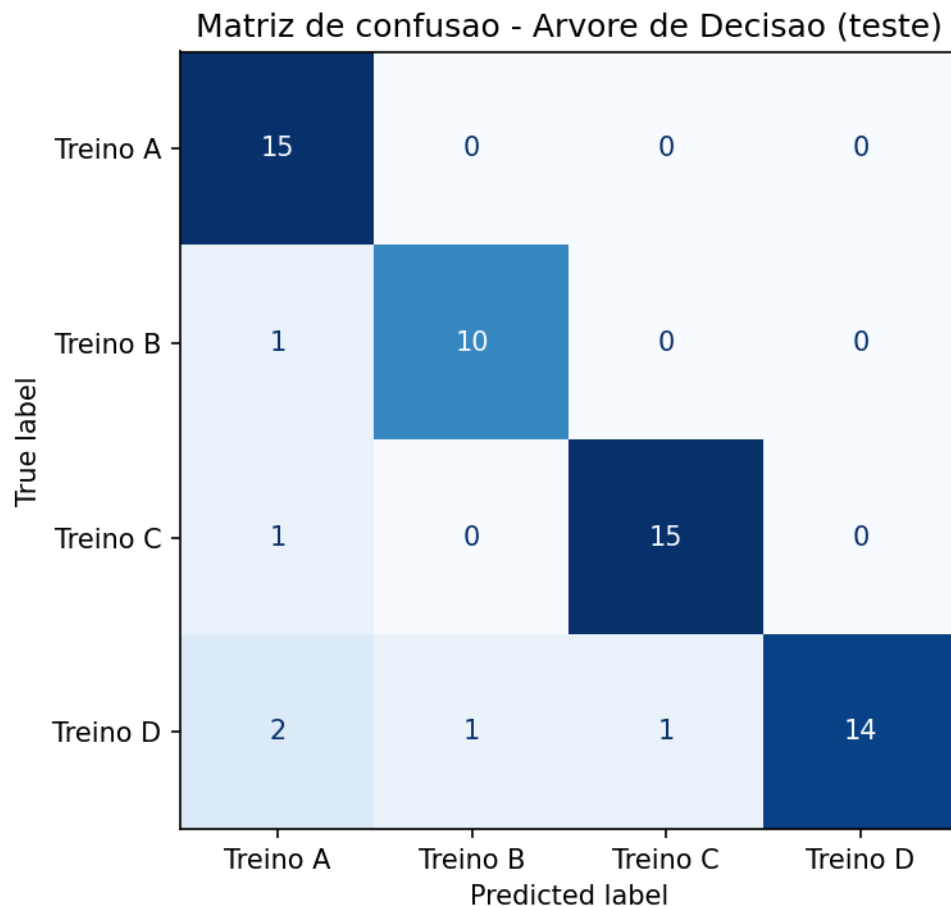
Como o problema tem 4 classes com tamanhos parecidos, reportamos classification_report com precisao, recall e F1 por classe, alem das medias macro e weighted.

Classe	Precisao	Recall	F1-score	Support
Treino A	0.79	1.00	0.88	15
Treino B	0.91	0.91	0.91	11
Treino C	0.94	0.94	0.94	16
Treino D	1.00	0.78	0.88	18
accuracy			0.90	60
macro avg	0.91	0.91	0.90	60

weighted avg	0.91	0.90	0.90	60
--------------	------	------	------	----

3. Matriz de confusao (figura)

Linhas = classe real, colunas = classe prevista.



Leitura: o Treino D e a classe mais dificil (recall 0,78, a menor das quatro), confundida com A, B e C em 4 dos 18 casos reais; as outras tres classes tem recall e F1 acima de 0,90.

4. Comparacao com o baseline

Baseline (DummyClassifier, most_frequent): 0,30

Modelo (Arvore de Decisao): 0,90

Diferenca: +0,60 - o modelo supera claramente o baseline.

5. Significado dos erros no dominio do projeto

Este e um problema multiclasse (Treino A, B, C, D), entao nao ha um unico par falso positivo/falso negativo global - cada classe tem o seu. O caso mais importante e o do Treino D (treino adaptado/peso corporal), usado quando o aluno nao tem equipamento, tem restricao fisica relevante ou e iniciante com pouca disponibilidade.

Falso negativo de Treino D (aluno precisava do treino adaptado, mas o modelo recomendou A, B ou C): na pratica, recomendar um treino mais intenso ou com mais carga do que o indicado

para alguém com restrição física, pouco equipamento ou pouca experiência - o erro mais caro do projeto, com risco real de lesão ou desistência.

Falso positivo de Treino D (aluno não precisava do treino adaptado, mas o modelo recomendou D mesmo assim): na prática, o aluno recebe um treino mais conservador do que poderia fazer - sub-aproveitado, mas sem risco à saúde.

6. Escolha e justificativa da métrica principal

A métrica principal escolhida é o recall da classe Treino D (0,7778) - quantos dos alunos que realmente precisavam do treino adaptado o modelo conseguiu identificar - e não a acurácia global.

Justificativa pelo custo do erro: um falso negativo de Treino D tem custo de segurança; um falso positivo de Treino D tem apenas custo de eficiência. Como errar para o lado perigoso é pior do que errar para o lado conservador, priorizamos recall sobre precisão para essa classe específica. O F1-macro (0,90) é reportado como visão agregada de apoio, mas a decisão sobre se o modelo está pronto para uso depende do recall de Treino D.

7. Teste de overfitting (treino vs. teste)

Acurácia no treino: 0,8875

Acurácia no teste: 0,90

Diferença (treino - teste): -0,0125

Leitura: a diferença é mínima e o teste ficou até levemente acima do treino - o oposto do que se esperaria de um modelo decorado. Isso é evidência de que não há overfitting: a profundidade limitada ($\text{max_depth}=5$) é suficiente para generalizar sem memorizar o ruído de rótulo identificado na Atividade 9.

8. Verificação de resultado bom demais

A acurácia de 0,90 já foi investigada na Atividade 9 (vazamento de dados e ruído de rótulo, 8% da base). Aqui, duas checagens adicionais:

Linhas com perfil (features) duplicado na base toda, antes do split: 14 de 300

Perfis (após dummies) idênticos presentes tanto no treino quanto no teste: 5

Vazamento de dados: `aluno_id` foi removido das features antes do treino, e nenhuma feature é calculada a partir do rótulo. Ordem cronológica: a base não tem nenhuma variável de tempo/data - os 300 alunos são simulados de forma independente, sem sequência temporal - logo a divisão aleatória estratificada é apropriada. Duplicatas: 14 das 300 linhas (4,7%) têm o mesmo perfil de features que outra linha, e 5 desses perfis aparecem tanto no treino quanto no teste (8,3% do conjunto de teste) - um efeito esperado numa base pequena com poucas variáveis categóricas, contribuinte plausível mas pequeno para a acurácia alta, não a explicação principal (que continua sendo a recuperação da regra de geração sintética, vista na Atividade 9).

9. Dificuldades encontradas

Entender a diferença entre métricas agregadas (acurácia, F1-macro) e métricas por classe, já que o risco real do projeto está concentrado em uma única classe (Treino D).

Traduzir falso positivo/falso negativo para a linguagem do problema (risco de lesão vs. treino sub-aproveitado) exigiu voltar à regra de geração (`gerar_dataset.py`).

O resultado do teste de overfitting foi uma surpresa: esperava-se treino maior que teste, e o oposto ocorreu - foi preciso entender que isso também é um sinal saudável, não um erro de cálculo.

Avaliar se os 5 perfis repetidos entre treino e teste invalidavam o resultado; conclusão: é um efeito esperado de uma base pequena, não um vazamento de dados.

10. Backlog atualizado para a AP2

[Feito] Treinar baseline e primeiro modelo (Atividade 9)

[Feito] Matriz de confusão, F1 por classe e verificação de vazamento (Atividade 9)

[Feito] Métrica principal escolhida e justificada pelo custo do erro (Atividade 10)

[Feito] Teste de overfitting treino vs. teste (Atividade 10)

[Feito] Verificação de duplicatas entre treino e teste e de ordem cronológica (Atividade 10)

[Feito] Slides finais da AP2 consolidados (entregas/AP2_Final_ViniciusCasagrande.pptx, 11 slides)

[Feito] Validação cruzada ($k=5$) - média 0,8267, desvio padrão 0,0291 (seção 12)

[Feito] Testar ajuste para melhorar o recall de Treino D - `class_weight='balanced'` levou o recall de 0,7778 para 0,8333, sem piorar as demais classes (seção 13)

[Falta] Aplicar `class_weight='balanced'` como configuração padrão do modelo (hoje só foi testado e comparado)

[Falta] Discutir o que o resultado de 0,90 significa para alunos reais, não apenas para a base sintética

[Falta] Preparar a defesa oral: explicar o recall de Treino D e o motivo da escolha dessa métrica

11. Conclusão

O modelo manteve acurácia de 0,90 no teste, bem acima do baseline (0,30) e da expectativa inicial (0,60-0,80). A métrica principal deste projeto não é a acurácia agregada, e sim o recall de Treino D (0,7778), porque o erro mais caro é recomendar um treino mais intenso do que o indicado - e esse recall é justamente o ponto mais fraco do modelo. O teste de overfitting não mostrou sinal de memorização, e a verificação adicional de resultado bom demais (duplicatas e ordem cronológica) não trouxe explicação nova preocupante. A leitura geral permanece a da Atividade 9: o modelo recupera bem a regra de geração sintética, mas isso não comprova desempenho equivalente com alunos reais.

Atualização na entrega final: as três investigações das seções 12 a 14 reforçam essa leitura com dados concretos. A validação cruzada (seção 12) mostra que a expectativa realista de desempenho está mais perto de 0,83 do que de 0,90. O teste de `class_weight='balanced'` (seção 13) mostrou que o recall de Treino D pode subir para 0,8333 sem custo aparente - uma recomendação concreta para antes da AP2. E a comparação das regras da árvore com a regra de geração (seção 14) mostra, caso a caso, onde o modelo erra: em pontos onde usa idade como atalho numa região que deveria depender de nível de experiência ou dias disponíveis, provavelmente por causa do ruído de rótulo.

12. Validação cruzada ($k=5$)

Pendente desde o pipeline da Atividade 7 e apontada pela professora no feedback do esboço. Em vez de confiar em um único split treino/teste, repetimos o treino e o teste 5 vezes em pedaços diferentes da base.

Acurácia por fold: [0.7833, 0.85, 0.85, 0.85, 0.8]

Média: 0.8267

Desvio padrao: 0.0291

Para comparar: acuracia do split unico usado nas secoes anteriores = 0.9

Leitura: a media dos 5 folds (0,8267) ficou abaixo do resultado do split unico (0,90), com desvio padrao de 0,0291 - cerca de 7 pontos percentuais de diferenca. O fold mais fraco (0,7833) ficou mais de 10 pontos abaixo do resultado reportado nas secoes anteriores. Isso mostra que o split original (random_state=42) foi relativamente favoravel, e que a expectativa realista para o modelo esta mais proxima da media de ~0,83 do que da acuracia do split unico - reforcando por que a decisao sobre o modelo nao deveria se apoiar so na acuracia agregada.

13. Teste de class_weight='balanced' para o recall de Treino D

Sugestao da professora: a base tem ~8% de ruído de rotulo proposital, entao existe um teto natural para qualquer metrica. Mesmo assim, testamos class_weight='balanced', que faz a arvore penalizar mais os erros em classes minoritarias durante o treino.

Recall de Treino D | original: 0.7778 | balanced: 0.8333

Acuracia geral | original: 0.9 | balanced: 0.9167

Leitura: class_weight='balanced' melhorou o recall de Treino D de 0,7778 para 0,8333 (+0,0556) e, surpreendentemente, tambem melhorou a acuracia geral, de 0,90 para 0,9167. A precisao de Treino A subiu junto (de 0,7895 para 0,8333), e nenhuma outra classe piorou - Treino B e Treino C ficaram identicos. Neste split de teste, a troca nao teve custo visivel. Isso nao elimina o problema - o teto de ruído de rotulo continua limitando o recall maximo possivel - mas e evidencia concreta de que balancear os pesos das classes e um ajuste simples que vale a pena adotar antes da AP2.

14. Comparando as regras aprendidas com a regra de geracao (gerar_dataset.py)

A arvore e interpretavel, entao em vez de so olhar o numero do recall, olhamos QUAIS regras ela aprendeu para prever Treino D e comparamos com a regra usada para gerar a base (funcao recomenda_treino).

Falsos negativos de Treino D no teste: 4

- idade 37, Hipertrofia, Iniciante, 2 dias, sem restricao, equip. completo -> previsto Treino A

- idade 41, Emagrecimento, Iniciante, 4 dias, restricao Lombar, equip. limitado -> previsto Treino A

- idade 16, Emagrecimento, Intermediario, 1 dia, sem restricao, equip. completo -> previsto Treino B

- idade 26, Hipertrofia, Intermediario, 1 dia, restricao Ombro, equip. completo -> previsto Treino C

Leitura: dos 4 falsos negativos de Treino D, nenhum tem equipamento 'Nenhum' (a condicao mais obvia da regra) - todos sao casos sutis, ligados a restricao fisica, nivel de experiencia e dias disponiveis. Comparando a arvore (export_text) com a regra real: em pelo menos dois casos a arvore aprendeu a usar IDADE como criterio de decisao numa regioao onde a regra real depende de nivel de experiencia ou de dias disponiveis - um atalho que funciona parcialmente

nos dados de treino, mas não é a regra verdadeira do domínio. Isso é consistente com os 8% de ruído de rótulo da base: a árvore, ao tentar se ajustar a esse ruído dentro do limite de profundidade 5, criou divisões por idade que não generalizam bem. Essa é uma limitação concreta para a AP2: o modelo erra mais onde regras baseadas em nível de experiência ou restrição física se cruzam com poucos dias disponíveis - exatamente os perfis de maior risco.

15. Organização, dificuldades e observações

O notebook `atividade10_academia.ipynb` foi organizado em 15 seções (markdown + código): reprodução do pipeline da Atividade 9, métricas de classificação, matriz de confusão, comparação com o baseline, significado dos erros no domínio, escolha da métrica principal, teste de overfitting, verificação de resultado bom demais, dificuldades, backlog da AP2, conclusão, validação cruzada, teste de `class_weight='balanced'` e comparação das regras aprendidas com a regra de geração. Foi executado do início ao fim no Google Colab e está disponível no repositório.

O que mudou em relação ao esboço: a análise original (seções 1 a 11) não mudou. Foram acrescentadas três investigações motivadas pelo feedback da professora sobre o esboço (nota 0,05/0,05): validação cruzada $k=5$, teste de `class_weight='balanced'` (melhorou o recall de Treino D de 0,78 para 0,83, sem custo aparente) e comparação das regras da árvore com a regra de geração, identificando onde o modelo usa idade como atalho em vez do critério real. A correção de nome de arquivo apontada no feedback (AP2_Preliminar -> AP2_Final) já estava alinhada no repositório. Os slides da AP2 foram consolidados de uma versão preliminar (8 slides) para a versão final (11 slides).

16. Declaração de uso de Inteligência Artificial

Uso de IA: Sim

Ferramentas utilizadas: Claude (Anthropic) e ChatGPT / IA generativa

Como foi usada:

apoio na estruturação do código das novas seções (validação cruzada, `class_weight='balanced'`, `export_text` e comparação com a regra de geração) e na organização do repositório; revisão da lógica de avaliação do modelo e da escolha da métrica principal; apoio na escrita do texto explicativo, na organização das seções da atividade e na revisão ortográfica;

revisão de conceitos de avaliação de modelos (overfitting, custo assimétrico de erro, verificação de vazamento, balanceamento de classes) aplicados ao problema.

Produção própria: a interpretação dos erros na linguagem do domínio do projeto, a escolha e justificativa da métrica principal pelo custo do erro, a leitura do teste de overfitting, a análise das duplicatas de perfil, a leitura crítica da validação cruzada e do teste de `class_weight='balanced'`, e a identificação do padrão de erro (uso de idade como atalho) na comparação das regras da árvore foram realizadas pelo aluno, com adaptação ao caso real do projeto.